Neural Networks

Analogy: natural learning

The brain is a highly complex, **non-linear, parallel** structure

It has an ability to organize its neurons to perform complex tasks

A neuron is 5/6 times slower than a logic gate

The brain overcomes slowness through a parallel structure

The human cortex has 10 billion neurons and 60 trillion synapses

What are neural networks?

(Artificial) Neural networks are models of machine learning that follow an **analogy** **with the functioning of the human brain**

A neural network is a **parallel** processor, consisting of simple processing units (neurons)

Knowledge is stored in the **connections** between the neurons

Knowledge is acquired from the environment (data) through a **learning process** (training algorithm) that **adjusts the weights** of the connections

Basic unit - Artificial neurons

Receive a set of inputs (data or connections)

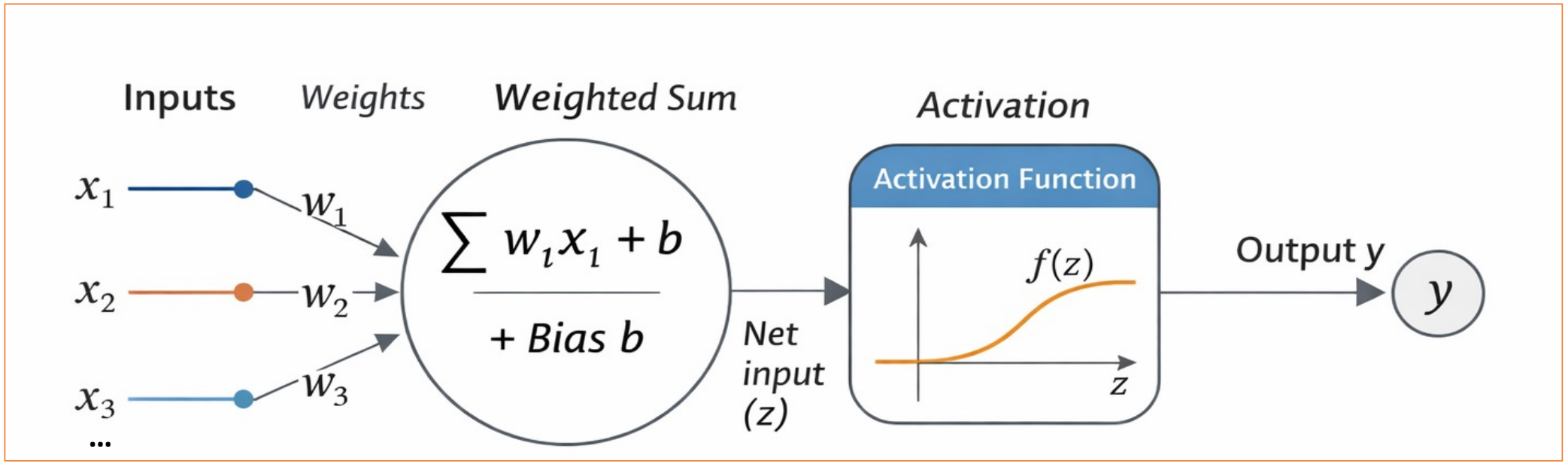
A **weight** (numerical value) is associated with each connection

Each neuron calculates its **activation** based on the input values and the weights of the connections

The calculated signal is passed on to the output after being filtered by an **activation function**

Structure of a neuron

$$z = w_1x_1 + w_2x_2 + w_3x_3 + \cdots + w_nx_n + b$$
$$y = f(z)$$



What model do you get if the activation function is the identity function ?

Activation functions

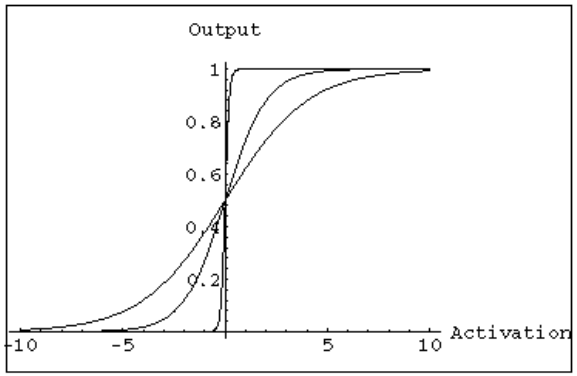
Sigmoid/Logistic

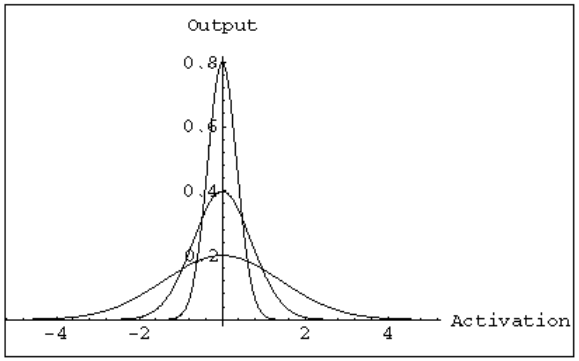
Linear

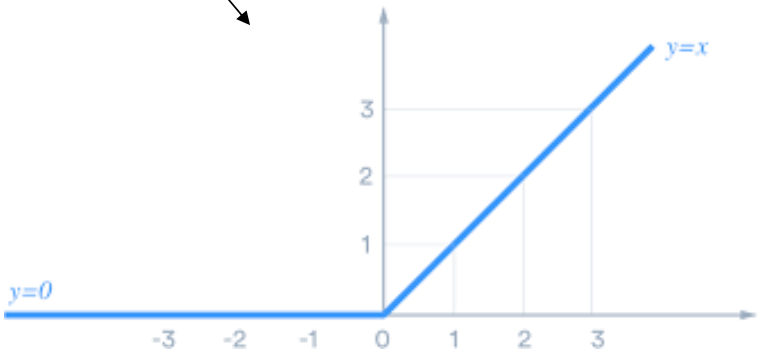
Hyperbolic tangent (Tanh)

Gaussian

RelU (Rectified linear)







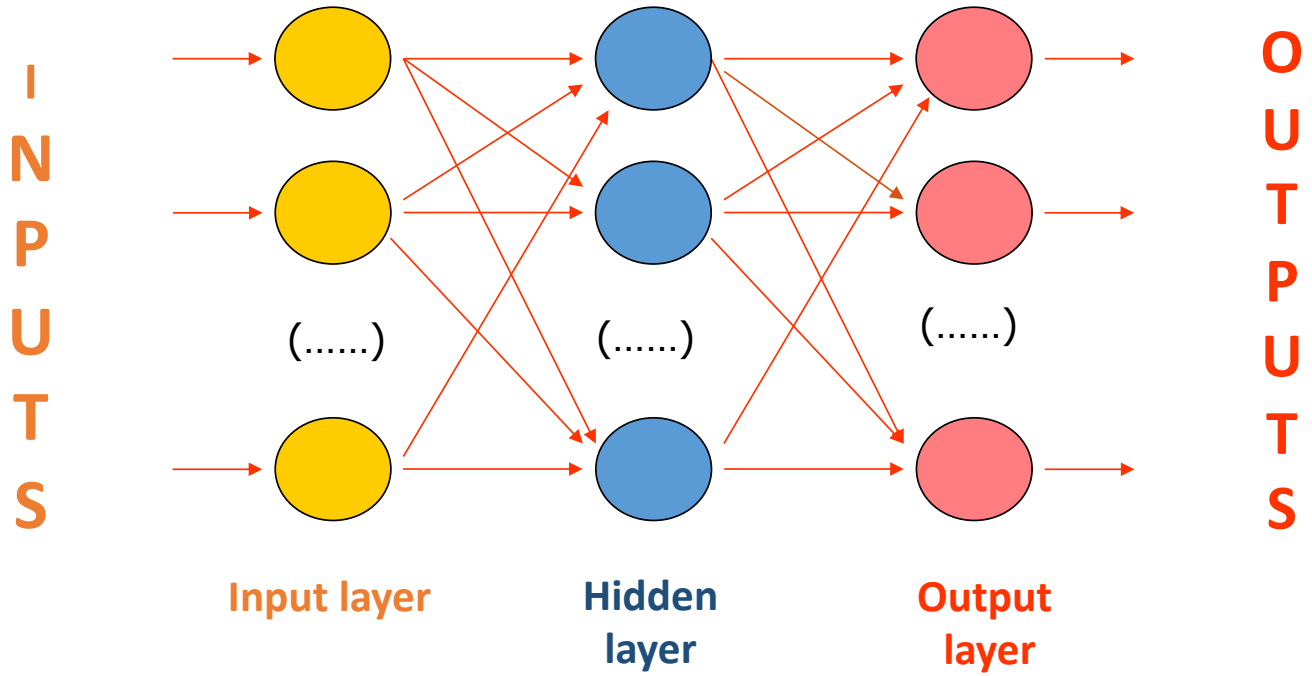
What model do you get if the activation function is the sigmoid function ?

Network topologies

Architecture (or **topology**) - the way nodes interconnect in a network structure (graph)

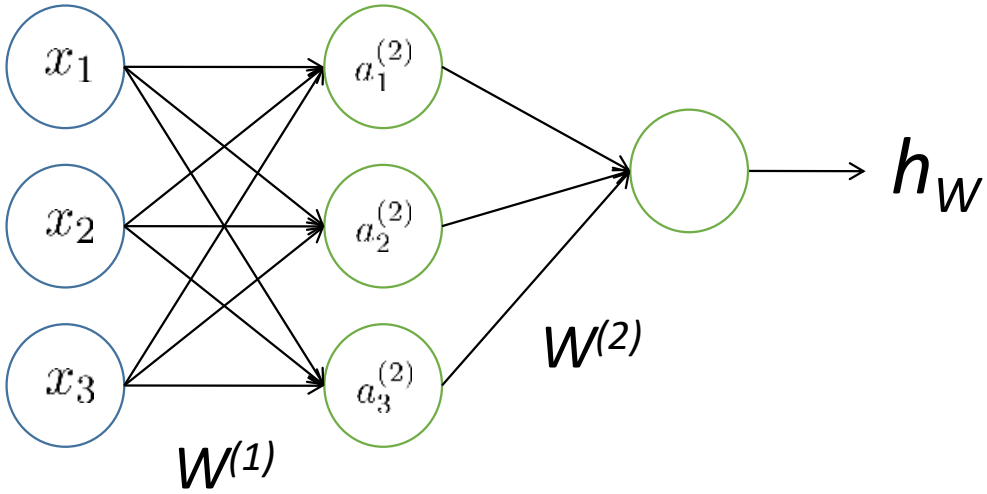
There are countless types of architectures, each with their own potentialities, falling into two categories: supervised and unsupervised, regarding the way they are trained

Feedforward neural network



Multilayer perceptrons (MLPs)

Neural network – computing output



$$a_1^{(2)} = f(b_1^{(1)} + W_{11}^{(1)}x_1 + W_{21}^{(1)}x_2 + W_{31}^{(1)}x_3)$$
$$a_2^{(2)} = f\left(b_2^{(1)} + W_{12}^{(1)}x_1 + W_{22}^{(1)}x_2 + W_{32}^{(1)}x_3\right)$$
$$a_3^{(2)} = f\left(b_3^{(1)} + W_{13}^{(1)}x_1 + W_{23}^{(1)}x_2 + W_{33}^{(1)}x_3\right)$$

$$h_W = a_1^{(3)} = f(b_1^{(2)} + W_{11}^{(2)}a_1^{(2)} + W_{21}^{(2)}a_2^{(2)} + W_{31}^{(2)}a_3^{(2)})$$

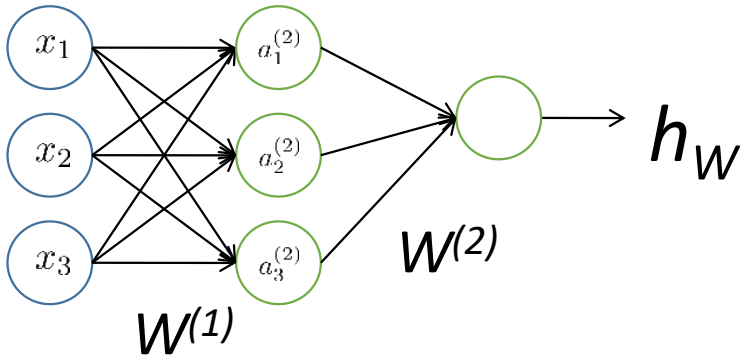
$a_i^{(j)}$ - "activation" of neuron *i* in layer *j*

$W^{(j)}$ -weight matrix for connections between neurons of layers *j* and *j+1* (rows – origin, columns - destination)

$b^{(j)}$ – biases of the neurons in layer j

The same process would apply to further layers

Neural network – computing output - vectorized



$$z^{(2)} = xW^{(1)} + b^{(1)}$$
$$a^{(2)} = f(z^{(2)})$$

$$z^{(3)} = a^{(2)}W^{(2)} + b^{(2)}$$
$$h_W = a^{(3)} = f(z^{(3)})$$

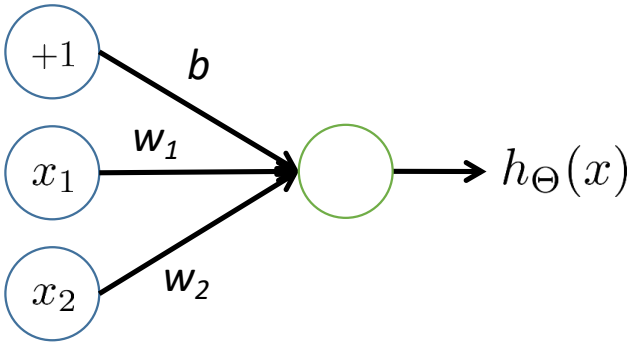
Output value

General version for layer i

$$z^{(i+1)} = a^{(i)}W^{(i)} + b^{(i)}$$
$$a^{(i+1)} = f(z^{(i+1)})$$

When processing several examples:
x becomes matrix **X**
$\boldsymbol{a^{(i)}}$ becomes a matrix $\mathbf{A^{(i)}}$
but the expressions are similar !

Computing the output - exercise



x_1	x_2	*Output*
0	0	
0	1	
1	0	
1	1	

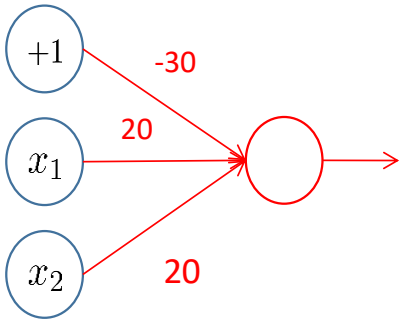
Calculate the output value for the cases where the weights are :

b = - 30, w_1 = 20, w_2 = 20

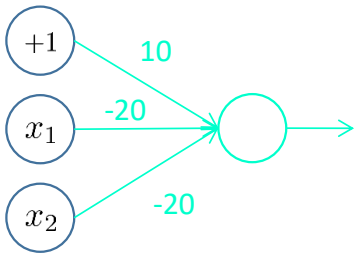
b = -10, w_1 = 20, w_2 = 20

b = 10, w_1 = -20, w_2 = -20

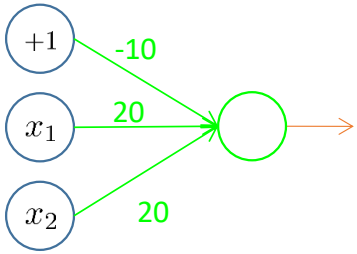
Computing the output - exercise



x_1 AND x_2



(NOT x_1) AND (NOT x_2)

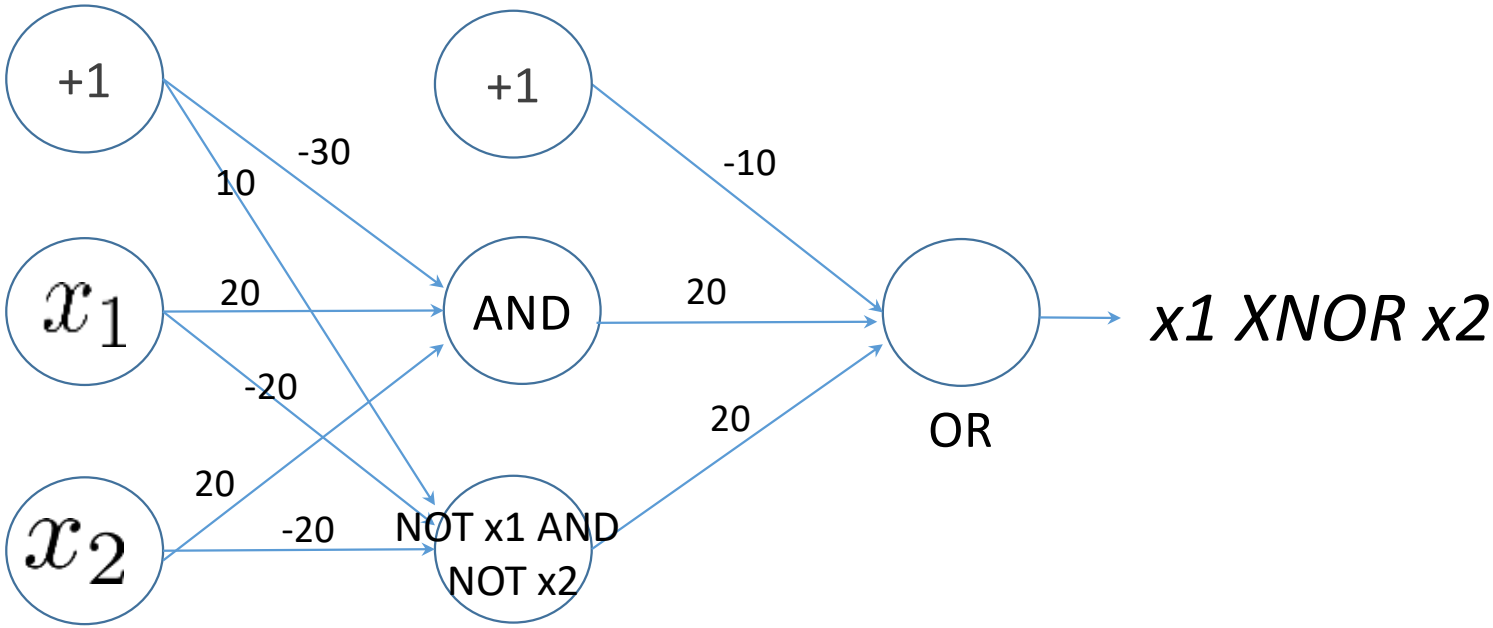


x_1 OR x_2

How to do H (x1, x2) = x1 XNOR x2 with a network with a hidden layer?

Note that: x1 XNOR x2 = (x1 AND x2) OR (NOT x1 AND NOT x2)

Computing the output - exercise



x1 XNOR x2 = (x1 AND x2) OR (NOT x1 AND NOT x2)

Pre-processing the data

Data standardization in neural networks is common given the used activation functions used; features with very different distributions of values are not convenient

Missing values in input features may be represented as zeros, which do not influence the neural net training process

Interpreting the outputs in classification

When using neural nets (and other functional models) to address multiclass classification datasets, we need to convert the output of the model (numerical value) into the discrete value (predicted class) that is desired.

As mentioned before, we will typically use one neuron per class, which implies to apply **one-hot encoding** to the output variable

In one-hot encoding, there are M output neurons (1 per class), being chosen for any case the class with the highest value.

Using the **softmax** activation function, we can get probabilities for all classes

NNs for the different supervised tasks

When using neural nets to address supervised learning tasks, three main types of cases arise, which demand different configurations of the network and the training process:

Problem type	Output layer	Activation function (output)	Loss function
Binary classification	One single node	Sigmoid	Binary Cross entropy
Multiclass classification	One node per class	Softmax (sigmoid in each node, normalized to sum 1)	Generalized cross entropy
Regression	One node	Linear (or RELU if only positive outputs are possible)	Mean Squared Error

Training: supervised learning

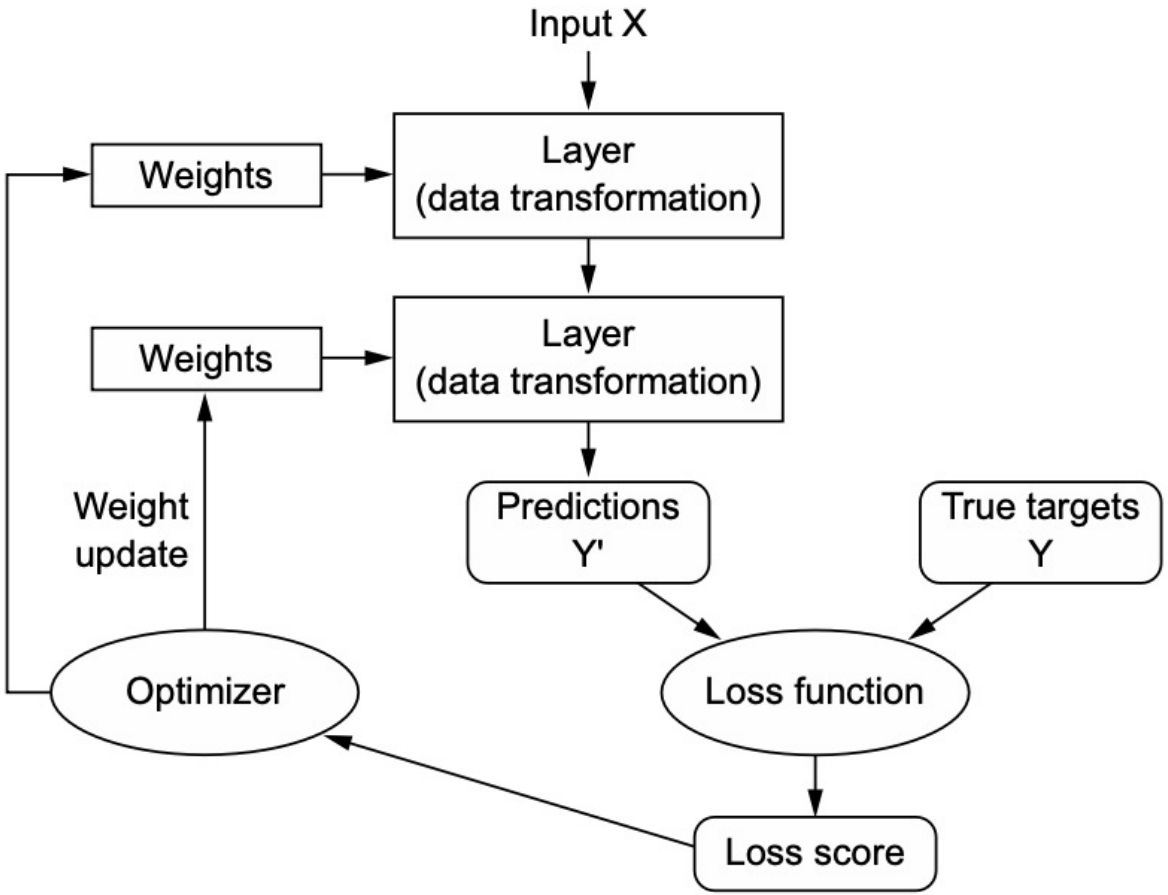
Data: Training examples consisting of **inputs** and their desired **outputs**

Objective: To set the values of the **connection weights** that minimize a cost function

Multiple **gradient-descent** algorithms exis:

- The most used historically is ***Backpropagation*** and derivatives
- Other algorithms: Marquardt-Levenberg, Rprop, Quickprop
- Most recent: RMSprop, Adam, SGD, Adagrad

Training: supervised learning



Overall scheme of the training/ learning process

Backpropagation steps

Forward propagation – calculates the output value(s) for the respective input vector and the error (given the known outputs);

Backpropagation – given the error on that sample, it is propagated back, adjusting the weights of the connections to reduce this error.

This process is based on the calculation of the **gradient** of the cost function, using the **chain rule** of partial derivatives for composite functions

This process is repeated multiple times in order to improve the network's performance.

Back-propagation algorithm

It is based on the gradient of the error surface that defines the direction of maximum descent (**gradient descent**)

Important parameter: **Learning rate-** sets the size of the steps in that direction

A sequence of these movements leads to a **minimum** (desirably global)

The training takes place over a number of **epochs** (iterations)

Initial network configuration (connection weights) - randomly generated

Stopping criterion: **fixed number of epochs**, elapsed time or convergence criterion based on a subset of validation examples

Training: chain rule and computation graphs

As we have already seen, a NN implements the **composition of functions (layers):**

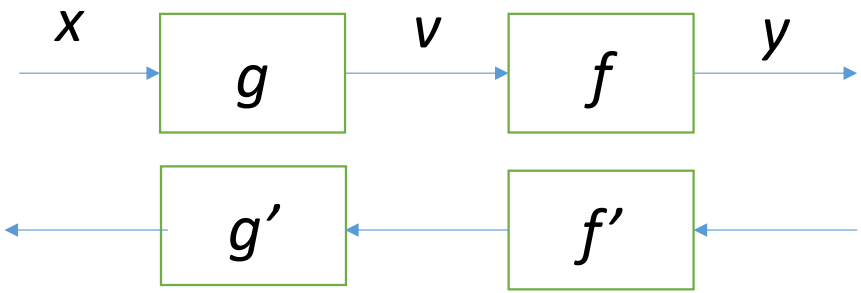
- from the calculation of the activation from the weights and inputs of each neuron
- through the application of the activation functions
- by the interconnection of neurons in successive layers, according to the topology of the network

Thus, the cost function of a NN includes many parameters (weights) defined at **various levels** in the network

Training: chain rule and computation graphs

To facilitate the calculation of the derivatives of this cost function, in relation to the weights to be optimized, the chain rule is used to calculate derivatives of composite functions

Computationally, the process of calculating the output of the NN and the cost function is organized in a **computation graph**, used for the calculation of derivatives by reversing the direction of calculation and allowing to work layer by layer



$$v = g(x)$$
$$y = f(g(x))$$

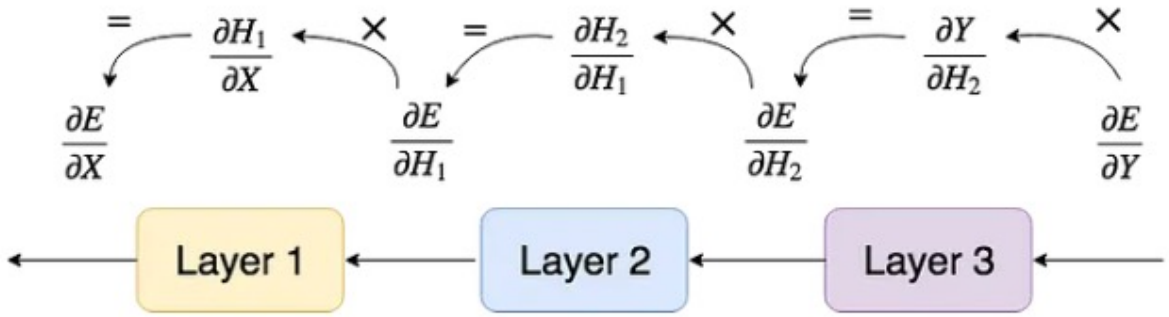
$$\frac{dy}{dx} = \frac{dy}{dv} \cdot \frac{dv}{dx}$$

Chain Rule

$$\frac{d}{dx}[f(g(x))] = f'(g(x))g'(x)$$

Training: example with feedforward NN

Forward propagation



Backward propagation

Training: example with feedforward NN: dense layer

$$X \rightarrow \boxed{\text{layer}} \rightarrow Y$$

Dense layer

$$y_j = b_j + \sum_i x_i w_{ij}$$

Forward propagation

$$X = \begin{bmatrix} x_1 & \dots & x_i \end{bmatrix} \quad W = \begin{bmatrix} w_{11} \dots w_{1j} \\ \vdots \quad \ddots \quad \vdots \\ w_{i1} \dots w_{ij} \end{bmatrix} \quad B = \begin{bmatrix} b_1 & \dots & b_j \end{bmatrix}$$

$$Y = XW + B$$

Training: example with feedforward NN: dense layer

$$X \rightarrow \boxed{\text{layer}} \rightarrow Y$$

Backward propagation

STEP 1: compute the layer input error for layer i (the output error from the layer i+1), **dE/dX**, to pass on to the layer i-1 (this will be the dE/dY of the next layer)

$$\frac{\partial E}{\partial X} = \left[\frac{\partial E}{\partial x_1} \quad \frac{\partial E}{\partial x_2} \quad \dots \quad \frac{\partial E}{\partial x_i}\right]$$

Chain rule

$$\frac{\partial E}{\partial x_i} = \frac{\partial E}{\partial y_1}\frac{\partial y_1}{\partial x_i} + \dots + \frac{\partial E}{\partial y_j}\frac{\partial y_j}{\partial x_i}$$
$$= \frac{\partial E}{\partial y_1} w_{i1} + \dots + \frac{\partial E}{\partial y_j} w_{ij}$$

$$\frac{\partial E}{\partial X} = \left[\left(\frac{\partial E}{\partial y_1} w_{11} + \dots + \frac{\partial E}{\partial y_j} w_{1j}\right) \quad \dots \quad \left(\frac{\partial E}{\partial y_1} w_{i1} + \dots + \frac{\partial E}{\partial y_j} w_{ij}\right)\right]$$
$$= \left[\frac{\partial E}{\partial y_1} \quad \dots \quad \frac{\partial E}{\partial y_j}\right] \begin{bmatrix} w_{11} & \dots & w_{i1} \\ \vdots & \ddots & \vdots \\ w_{1j} & \dots & w_{ij} \end{bmatrix}$$
$$= \frac{\partial E}{\partial Y} W^t$$

Training: example with feedforward NN: dense layer

$$X \rightarrow \boxed{\text{layer}} \rightarrow Y$$

Backward propagation

STEP 2: compute the weights' errors: **dE/dW** = X.T * dE/dY

$$\frac{\partial E}{\partial W} = \begin{bmatrix} \frac{\partial E}{\partial w_{11}} & \cdots & \frac{\partial E}{\partial w_{1j}} \\ \vdots & \ddots & \vdots \\ \frac{\partial E}{\partial w_{i1}} & \cdots & \frac{\partial E}{\partial w_{ij}} \end{bmatrix}$$

Chain rule

$$\frac{\partial E}{\partial w_{ij}} = \frac{\partial E}{\partial y_1}\frac{\partial y_1}{\partial w_{ij}} + \ldots + \frac{\partial E}{\partial y_j}\frac{\partial y_j}{\partial w_{ij}} = \frac{\partial E}{\partial y_j} x_i$$

$$\frac{\partial E}{\partial W} = \begin{bmatrix} \frac{\partial E}{\partial y_1}x_1 & \cdots & \frac{\partial E}{\partial y_j}x_1 \\ \vdots & \ddots & \vdots \\ \frac{\partial E}{\partial y_1}x_i & \cdots & \frac{\partial E}{\partial y_j}x_i \end{bmatrix} = \begin{bmatrix} x_1 \\ \vdots \\ x_i \end{bmatrix} \begin{bmatrix} \frac{\partial E}{\partial y_1} & \cdots & \frac{\partial E}{\partial y_j} \end{bmatrix} = \boxed{X^t \frac{\partial E}{\partial Y}}$$

Training: example with feedforward NN: dense layer

$$X \rightarrow \boxed{\text{layer}} \rightarrow Y$$

Backward propagation

STEP 3: compute the biases' errors: **dE/dB** = dE/dY

$$\frac{\partial E}{\partial B} = \begin{bmatrix} \frac{\partial E}{\partial b_1} & \frac{\partial E}{\partial b_2} & \cdots & \frac{\partial E}{\partial b_j} \end{bmatrix}$$

Chain rule

$$\frac{\partial E}{\partial b_j} = \frac{\partial E}{\partial y_1}\frac{\partial y_1}{\partial b_j} + \cdots + \frac{\partial E}{\partial y_j}\frac{\partial y_j}{\partial b_j} = \frac{\partial E}{\partial y_j}$$

$$\frac{\partial E}{\partial B} = \begin{bmatrix} \frac{\partial E}{\partial y_1} & \frac{\partial E}{\partial y_2} & \cdots & \frac{\partial E}{\partial y_j} \end{bmatrix} = \boxed{\frac{\partial E}{\partial Y}}$$

Training: example with feedforward NN: dense layer

$$X \rightarrow \boxed{\text{layer}} \rightarrow Y$$

Backward propagation

STEP 4: update weights and biases using gradient descent

$$w_i \leftarrow w_i - \alpha \frac{\partial E}{\partial w_i}$$

$$b_i \leftarrow b_i - \alpha \frac{\partial E}{\partial b_i}$$

Training: activation functions

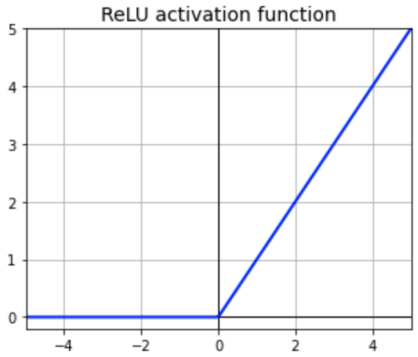
Activation functions will work similarly to other layers, being defined their forward and backpropagation computation graphs

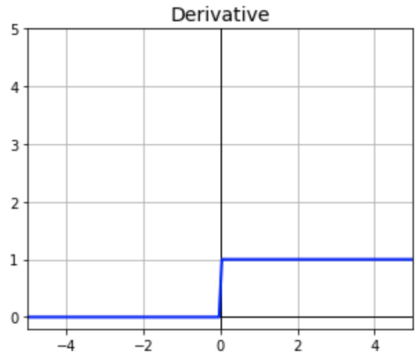
Since they do not have weights, only the error propagation step (step 1) is done for these cases

RelU

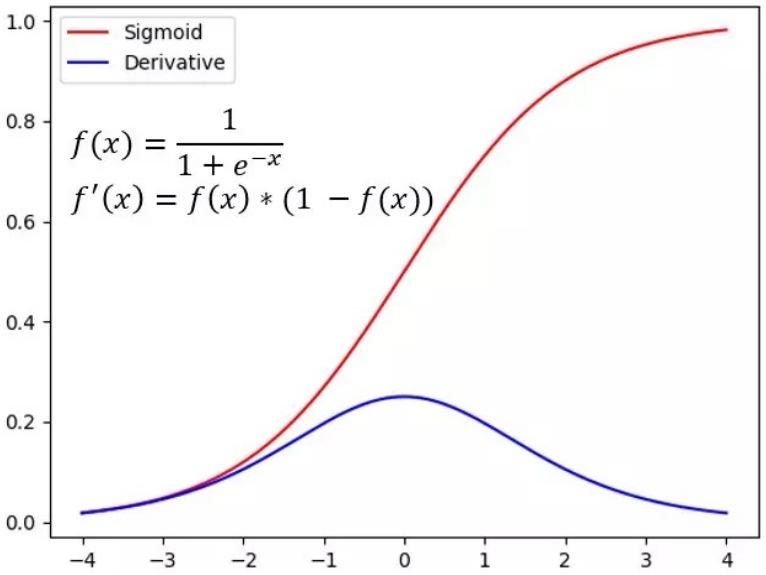
$$f(x) = \begin{cases} 0 & \text{if } x < 0 \\ x & \text{if } x \geq 0 \end{cases}$$

$$f(x) = \begin{cases} 0 & \text{if } x < 0 \\ 1 & \text{if } x \geq 0 \end{cases}$$



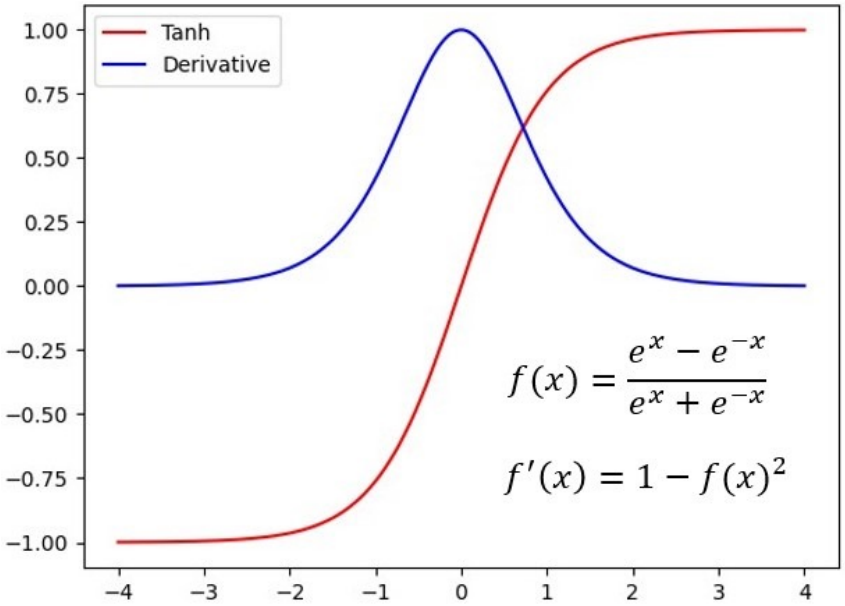


Sigmoid



Training: activation functions

Tanh



Softmax

$$f(x_i) = \frac{e^{x_i}}{\sum_j^n e^{x_j}} = \frac{e^x}{\sum e^x}$$

$$f'(x) = f(x)(1 - f(x))$$

Training: cost function

The cost function is the one that will be minimized in backpropagation

Used functions in NN are similar to those used in linear and logistic regression, being the sum of the error function for the set of individual examples

The error function (***loss function***) for each example will typically be given by:

- Sum/ mean of the squared errors (SSE/ MSE), for regression problems
- (Binary) cross entropy (used in logistic regression) for binary classification
- A generalization of the previous one to handle multiple classes

Training: loss functions in backprop

Mean squared error

$$L = \frac{1}{n}\sum_{i}^{n}\left(y_i - y_i^*\right)^2$$

$$\frac{\partial L}{\partial Y} = \begin{bmatrix} \frac{\partial L}{\partial y_1} & \cdots & \frac{\partial L}{\partial y_i} \end{bmatrix}$$

$$= \frac{2}{n}\begin{bmatrix} y_1^* - y_1 & \cdots & y_i^* - y_i \end{bmatrix}$$

$$= \frac{2}{n}(Y^* - Y)$$

Training: loss functions in backprop

Binary cross entropy

$$L = -\frac{1}{n}\sum_{i=1}^{n}(y_i \log(y_i) + (1 - y_i)\log(1 - y_i))$$

$$\frac{\partial L}{\partial Y} = \frac{-Y}{Y^*} - \left(\frac{1 - Y}{1 - Y^*} * -1\right)$$

$$= \frac{-Y}{Y^*} + \frac{1 - Y}{1 - Y^*}$$

Categorical cross entropy

$$L = -\sum_{i=1}^{i=N} y_i \log(y_i^*)$$

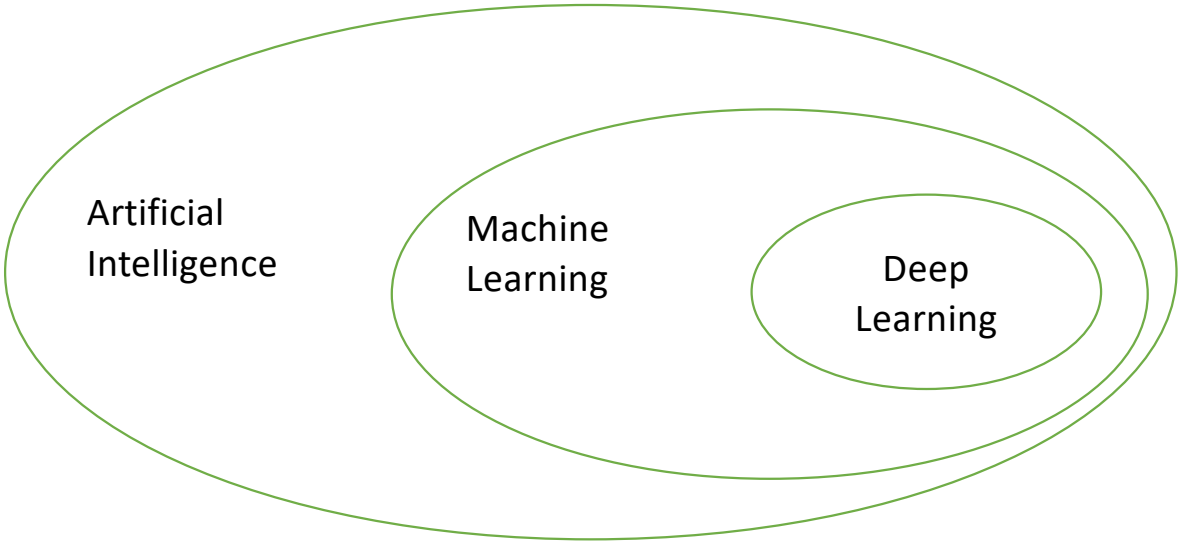
$$\frac{\partial L}{\partial Y} = -\frac{Y}{Y^*}$$

Deep neural networks

Deep learning: what is it?

Machine Learning field characterized by a greater complexity of models and the ability to learn representations from input data

Deep learning models consist of successive layers of representations, in a number that is typically high (deep models)



Deep learning: what is it?

Used models are typically based in neural networks structured in several processing layers

Different types of neurons and architectures used for different types of problems: feedforward neural networks, recurrent networks, convolutional networks, etc.

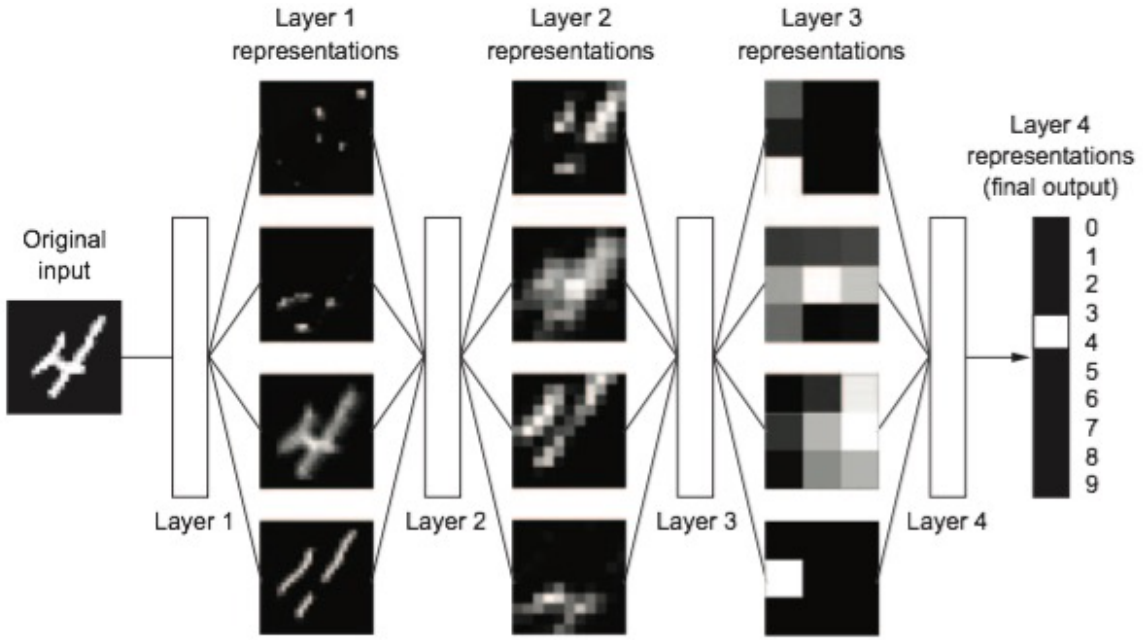
There are deep learning models for supervised and unsupervised learning problems, as well as reinforcement learning

Deep learning: what is it?

Several layers process information by creating distinct and typically more abstract representations of the inputs

In the case of supervised learning, the last layer represents the output of the neural net

Learning by **gradient descent** methods (next session)



Deep learning: major factors

Like any technology, DL does not solve all problems and will not always be the best option for any learning task

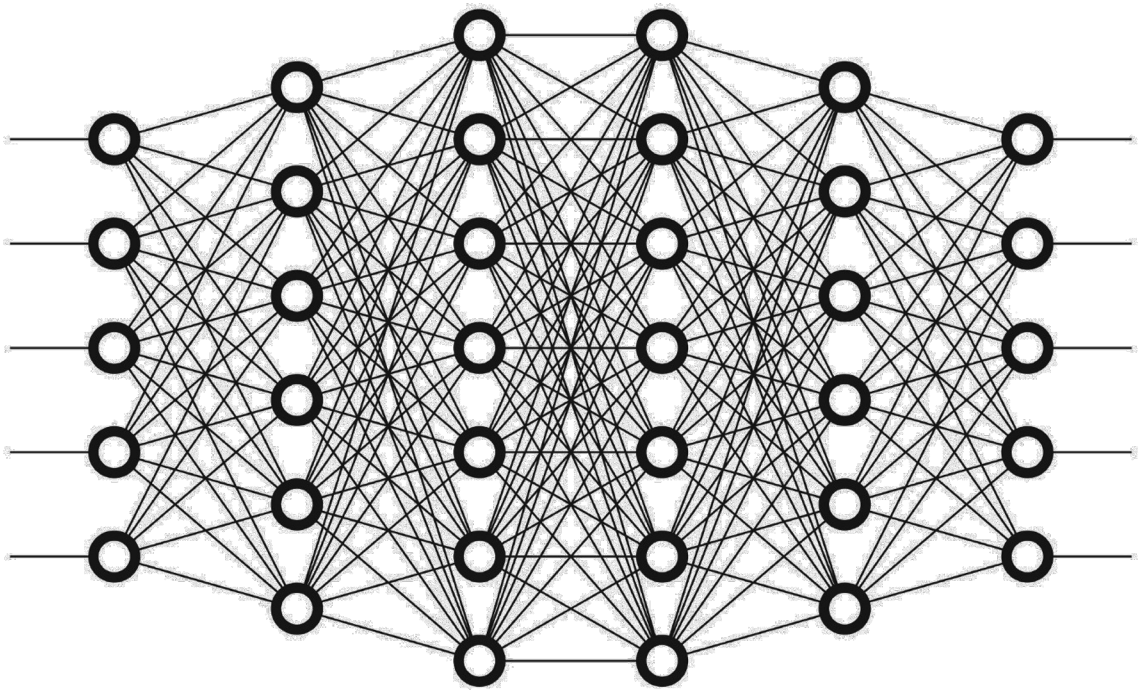
A determining factor for DL success is the availability of large-scale datasets; for problems with little data, other models can give more consistent results with less computational effort

In terms of hardware, the use of graphics processors (GPU) brings advantages in training DL models by accelerating the process by factors of more than 10x

Improvements in relation to "shallow" neural networks: activation functions (RelU), weight initialization, optimization algorithms (RMSprop, Adam), methods for addressing overfitting, pre-training, and training "by layers"

Deep neural networks (DNNs)

DNNs are supervised DL models, being **feedforward** NNs with typically several hidden layers



Deep Learning frameworks

Tensorflow from **Google**

- Multiple API levels - **Keras** official high-level API, user-friendly and enabling rapid prototyping. Defining a model with Keras often feels like describing a sequence or graph of layers. Common architectures very straightforward to implement
- TensorFlow 2.x adopted "eager execution" by default, making it behave more dynamically like Python code, similar to PyTorch.
- Good support for production deployment (TensorFlow Serving, TensorFlow Lite for mobile/embedded devices, TensorFlow.js for web), scalability across distributed systems
- Powerful visualization tools via TensorBoard
- Some installation problems are common

Improvements on training algorithms: momentum

Recent algorithms use **momentum** – calculating moving average of the error gradients for the last N updates

Way to remove update noise, allowing to use higher learning rates and accelerate the convergence of the training process

Adds a new parameter (β) defining the weight to the previous value (new batch contributes with 1 – β) – typical value of 0.9

Retained gradient $\longrightarrow V_t = \beta V_{t-1} + (1-\beta)\frac{\partial L}{\partial W}$

$$W_t = W_{t-1} - \alpha V_t$$

Improvements on training algorithms: RMSProp e Adam

RMSProp also uses moving averages, but here of the squared errors (uses also a parameter β – typical value of 0.999)

Weight update divides the derivative by the square root of the moving average (adding a small value ε to avoid zero values)

Adam combines momentum with the strategy of RMSProp, so having two parameters for the update - β1 e β2 - and also ε

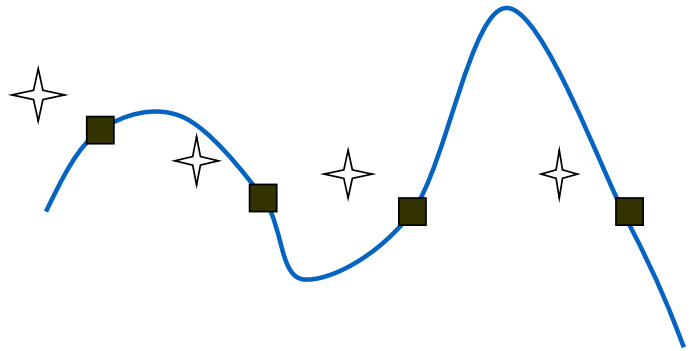
Overfitting and generalization in NNs

To train the network many epochs (training iterations) can prevent generalization and promote *overfitting*

It is preferable to stop the training in these cases – **early stopping**; error is measured in a validation dataset; if the validation error improves for a number of consecutive epochs, the training is stopped

The probability of overfitting increases if:

- Few training cases
- Many weights (complexity)



Overfitting and generalization

In DL, overfitting has been addressed using various techniques, such as **regularization** (L1 or L2, similar to linear/logistic regression, called **decay** in NNs), or explicit control of model **capacity** (e.g. number of hidden layers and number of neurons in each)

A specific technique developed for DL models is **dropout**: in each iteration of the training the output of some randomly selected neurons is put to zero. The proportion of zeroed neurons in each iteration is a parameter defined by each layer where dropout applies

Deep Learning frameworks

PyTorch by **Meta AI**

- **API:** "Pythonic" feel. It integrates tightly with the Python language and its ecosystem (e.g., NumPy). Defining models and custom operations feels like writing standard object-oriented Python code.
- Primarily uses dynamic computation graphs (define-by-run) - built on-the-fly as the code executes. This offers greater flexibility, especially for models with dynamic structures (common in natural language processing) and makes debugging more straightforward.
- Widely adopted in the research community due to its flexibility and ease of use. Rapidly growing ecosystem and adoption in production environments.
- We will be the main framework for the examples in this course

Improvements on training algorithms: batches

Doing the training process using **mini-batches** (batches with part of the examples) instead of considering all available examples to calculate errors and their derivatives and update weights

Batch sizes to consider change typically between 64 and 512; *batch* should be of a size to allow keeping it as a whole in GPU memory

Batch size can be tuned/ optimized

If batch = 1 -> ***stochastic gradient descent***

If batch = number of examples in the dataset -> ***batch gradient descent***